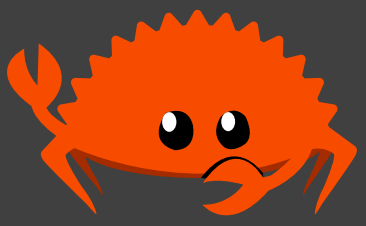

Rust Programming

Skylar Coelho

February 11, 2026

Contents

1 Rust Basics

1.1 Installing Rust

Install Rust from the Rust website. Once you have rust installed, check that the Rust compiler is set up correctly by running

```
$ rustc --version
```

Rust can be updated using rustup, the included tool chain for installing and maintaining Rust. The version of rustup can be checked in a similar matter. Rust can be updated via rustup by running

```
$ rustup --version
$ rustup update
```

1.2 Cargo Basics

Rust runs typically from a main function created by initializing the project using cargo. Initialize a project using cargo

```
$ cargo new project_name
```

If the directory already exists, you can build a project in that directory using

```
$ cargo init
```

You can add dependencies, like external libraries from crates.io, by running

```
$ cargo add package_name
```

Finally, when you're ready, you can build your project for debugging or release using cargo as well via

```
$ cargo build # compiles for debug by default
$ cargo build --release # for building using the release profile
$ cargo run # compiles for debug and runs the executable
$ cargo run --release # compiles and runs the release profile executable
```

Additionally, you can always compile your Rust script using the Rust compiler itself and then running the produced executable

```
$ rustc main.rs # compile
$ ./program_name # run the produced executable
```

1.3 Basic Setup

Typical Rust programs will contain the following code snippet within `main.rs` by default

```
fn main(){
    println!("Hello, world!")
}
```

The code contained in the main function is what gets compiled and called by the typical Rust compilation actions. Note that the default function setup contains the macro `println!` which prints the string literal `"Hello, world!"` to the console. We know this is a macro because the standard naming scheme for macros in Rust is to leave an "!" at the end of a function name.

1.4 Including Libraries

Certain libraries are packaged with Rust, known as the *standard library*, shortened to *std*. These packages do not need to be included as dependencies via cargo and can be brought into your code via the following syntax

```
use std::io
```

The double colon '::' is similar to the dot '.' notation from other languages where the above brings in the `io` library from the standard library.

1.5 Variable Declarations

New variables are declared using `let`. Variables are *immutable* by default and must be declared as *mutable* using the keyword `mut` during the variable declaration

```
let x = 5; // x is an immutable variable
let mut y = 6; // y is a mutable variable
```

You can, and should, annotate the type of a variable when declaring it like this

```
let x: i32 = 297;
let y: &str = "hello"
```

Note that `&str` is a reference to the string type which will be touched on later.

1.6 User Input

Input from the terminal can be obtained using the standard library's `io::stdin()` method.

```
io::stdin()
    .read_line(&mut guess);
```

Where a mutable reference to the variable ”guess” is passed to the standard io method allowing it to pass the terminal input to the location in memory where the variable guess is stored. This reference must be mutable because we are changing the value stored at that location. We can add the `except` method to this call and add a response in the case that the method fails.

```
io::stdin()
    .read_line(&mut guess)
    .except("Failed to read input");
```

2 Variables and Typing

Constants are declared with the keyword `const` and like most variables are immutable. The difference is that constants can *never* be made mutable.

```
const varname = 5;
```

Variables can also be reassigned via *shadowing* where an immutable variable has its value reassigned by re-declaring it using the current value with some modification added

```
let x = 5 // x is an immutable variable
let x += 5 // x now equals 10! We have changed an immutable variable
```

Note that this allows us to both change a variables value **and** its type because we are effectively creating a new variable with the same name. let number = if condition 5 else 6 ;

2.1 Scalar Data Types

Scalar data types contain data of only one type and fall into one of the following four categories

- Integers
- Floating Point Numbers
- Booleans
- Chars

2.1.1 Integers

Integers are whole numbers without decimal components and can be defined based on their size Numeric literals that can be interpreted as multiple types can be specified by annotating them within their use

```
    let x: i32 = 23 + 16u16
```

Numeric literals can be specified in any of the following formats If you’re unsure what to use, Rust’s default integer type is i32.

Table 1: Integer Types in Rust

Length	Signed	Unsigned	Size of Values
8-bit	i8	u8	$2^8 = 256$
16-bit	i16	u16	$2^{16} = 65,536$
32-bit	i32	u32	$2^{32} \cong 4.294 \times 10^9$
64-bit	i64	u64	$2^{64} \cong 18.446 \times 10^{18}$
128-bit	i128	u128	$2^{128} \cong 3.40 \times 10^{38}$
Architecture-dependent	isize	usize	See above

Table 2: Integer Literals in Rust

Number Literal	Example
Decimal	98_222
Hex	0xff
Octal	0o77
Binary	0b1111_0000
Byte (u8 only)	b'A'

2.1.2 Floating Points

Floating point numbers can be specified as `f32` or `f64` and are represented according to IEEE-754.

2.1.3 Booleans

Booleans are values that can either be true or false, and are annotated as `bool`. Note that Rust, unlike some other languages, will not interpret any non boolean type values as a bool. If the condition of non boolean type data is checked, it will fail.

2.1.4 Chars

Chars are a 4-byte type representing a Unicode scalar value.

2.2 Compound Types

Compound types are data types which themselves may contain multiple different pieces of data which may be different types of their own

2.2.1 Tuples

Tuples are types of fixed length which may contain multiple different data types. Note that once a tuple is declared, its length may not be increased or decreased, though it may be de-structured into individual pieces

```
    // declaring a tuple of different types
    let tup: (i32, f64, u8) = (500, 6.4, 1);

    // de-structuring a tuple
    let (x, y, z) = tup;
```

You may also access individual elements of a tuple using dot notation

```
    // declaring a tuple of different types
    let tup: (i32, f64, u8) = (500, 6.4, 1);

    // retrieving the first value from a tuple
    let x = tup.0;
```

Empty tuples with no type have their own name and type, *unit*, written as (). Any expression which does not inherently return a value will return unit.

2.2.2 Arrays

Arrays hold collections of multiple items which have the *same* type. Like tuples, arrays have a fixed length. Arrays are declared with brackets and only need one type declaration along with their length.

```
    // declaring an array of type i32 and length 5
    let a: [i32; 5] = [1, 2, 3, 4, 5];
```

Array values are accessed using bracket notation

```
    let first = a[0];
    let second = a[1];
```

3 Control Flow

Control flow is a basic and essential part of code which allows for controlling the path a written section of code takes based on designated inputs and information. Looping and conditional execution are two common examples of control flow.

3.1 Loop

Basic looping can be accomplished with the `loop` keyword which establishes a loop that continues until `break` is run

```
loop 'loopname {
    println!("Looping!");
    break 'loopname
}
```

Note that loops can also be named or "disambiguated" and continued or broken by their name. This is especially useful for controlling nested loops.

3.2 If Expressions

A true/false condition combined with the `if` keyword creates an *if expression* which allows you to branch into a piece of code if the statement is true.

```rust
if var == 5 {
    println!("True!")
}
```

For conditions with multiple branches, `else if` statements can be used to evaluate multiple clauses

```rust
let number = 6;
if number % 4 == 0 {
println!("number is divisible by 4");
} else if number % 3 == 0 {
println!("number is divisible by 3");
} else if number % 2 == 0 {
println!("number is divisible by 2");
} else {
println!("number is not divisible by 4, 3, or 2");
}
```

If statements are useful for more than just controlling the flow of a program. Another example is for variable assignment

```rust
let number = if condition { 5 } else { 6 };
```

Note that if the options are not the same type, you will likely run into issues.

3.3 Returning Values from Loops

Loops can also return a value by assigning the returned value of the loop to a variable

```rust
let mut counter = 0;
let result = loop {
    counter += 1;
    if counter == 10 {
    break counter * 2;
    }
};
```

3.4 For Loops

For collections like tuples and arrays, we can simply loop through each element using a `for` loop

```rust
let a = [10, 20, 30, 40, 50];

for element in a {
    println!("the value is: {element}");
}
```

A range can also be created using '..' notation like this

```
for k in (1..10) {
println("{k}");
}
```

4 Functions

Functions are the main way of executing code in Rust and are declared with the `fn` keyword. The usual entry point of a Rust program is the `main` function that is created with a new project by default

```
fn main() {

}

fn secondary(x: i32) -> i32 {
    x + 23
}
```

Functions can take inputs specified during the function signature of the annotated type and return a value of the specified type at the end of the function signature. Note that above, the value $x + 23$ is returned because it is the last expression in the function. A value elsewhere in the function can be returned using the keyword `return` followed by an expression.

This brings up the concept of expressions versus statements. The difference between the two is that expressions return a value while statements do not. Statements are terminated with a semicolon, while expressions are not.

5 Ownership

Ownership and the borrow checker is how Rust manages memory. The most important thing to know in this regard, is that variables are valid when they come in to scope and remain valid until they go out of scope.

6 Structs

Structs are Rust's answer to structured and labeled data. Define a struct using the keyword `struct`

```
struct User {
    active: bool,
    username: String,
    email: String,
    sign_in_count: u64,
}
```

Then instantiate an instance of the struct by defining it using `let`

```
let user1 = User {
    active: true,
    username: String::from("someusername123"),
    email: String::from("someone@example.com"),
    sign_in_count: 1,
};
```

Values are set and retrieved using dot notation

```
user1.email = String::from("anotheremail@example.com");
```

Note that a struct as a whole must be mutable for us to set these values after it is defined. We cannot set certain fields to be mutable and some not. If we initialize a struct using variables that match the field names, we can leave them out while initializing

```
User{
    username,
    email,
}
```

This is called *field init shorthand*.

If we want to initialize a struct with some, or all, of the fields equal to another instance of that struct, we can use *struct update syntax*

```
let user2 = User {
..user1
}
```

Tuple structs are a way to create a new type of tuple and give it named parameters

```
struct Color(i32, i32, i32);
struct Point(i32, i32, i32);
fn main() {
    let black = Color(0, 0, 0);
    let origin = Point(0, 0, 0);
}
```

Both of these tuples above are composed of three 32 bit integer values, but they are not the same type and functions that accept one will not accept the other. Structs that do not have any fields are called *unit structs* as they are similar to the unit type `()`. These can be used when we want to assign a trait to something without that trait having a value itself.

6.1 Printing Structs

By default, calling the `println!` macro on a struct will not work. We can get around this by asking Rust to print debug information on our struct via `:?`. Note that in order for our struct to provide debug information, it needs the outer attribute `#[derive(Debug)]` named prior to the struct definition.

```
#[derive(Debug)]
struct Rectangle {
    width: u32,
    height: u32,
}
fn main() {
    let rect1 = Rectangle {
        width: 30,
        height: 50,
    };
}
```

6.2 Methods

Methods are functions declared using an *implementation* of a struct. Declare a method within an `impl struct_name` block just as you would a regular function though the first parameter is always `&self`.

6.2.1 Associated Functions

If a function is implemented within an `impl` block, it is known as an *associated function*, as it is *associated* with a struct. If a function references the struct itself via `&self` it is further known as a method. Functions do not have to be methods however, and can be declared as an implementation without this reference.

```
impl Rectangle {
    fn square(size: u32) -> Self
    { 1
        2 Self {
            width: size,
            height: size,
        }
    }
}
```

These functions are called using the struct name and "::" syntax, like `String::from("string")`

7 Enums and Pattern Matching

Enums allow specification of a list of allowable values. Note that an *enum* value can only be one of the possibilities. To enumerate the possible values for IP addresses we would

```
enum IpAddrKind {
    V4,
    V6,
}
```

Make a value for kind of IP address

```
let four = IpAddrKind::V3;
let six = IpAddrKind::v6;
```

We could then make a function that only accepts types of ip address

```
fn route(ip_kind: IpAddrKind) {}
```

This function could be called on either variant of IP address. We can also create associated values in our enum definition

```
enum IpAddr {
        V4(String),
        V6(String),
}
let home = IpAddr::V4(String::from("127.0.0.1"));
let loopback = IpAddr::V6(String::from("::1"));
```

Note that defining this enum has also created a constructor function to define instances of an enum. We don't have to stop at one value though!

```
enum IpAddr {
        V4(u8, u8, u8, u8),
        V6(String),
}
let home = IpAddr::V4(127, 0, 0, 1);
let loopback = IpAddr::V6(String::from("::1"));
```

Enum values can also be named

```
enum Message {
        Quit,
        Move { x: i32, y: i32 },
        Write(String),
        ChangeColor(i32, i32, i32),
}
```

7.1 The Option Enum

The option enum is provided by the standard library and is utilized to represent that a value may or may not exist. For example, if you request the first value of a list, that value might not exist, but we still want an option to handle this scenario. Rust does not have nulls, but it does have `Option<T>` as defined by the standard library

```
enum Option<T> {
        None,
        Some(T),
}
```

These are so pervasive that they are included in Rust's prelude, that is that you don't even have to manually include them in scope.

```
let some_number = Some(5);
let some_char = Some('e');

let absent_number: Option<i32> = None;
```

Above, the variables are of type `Option<i32>`, `Option<char>`, and `Option<i32>` respectively. Option was able to infer the first two but cannot infer the type of none so it was annotated in the variable definition. Note that `i32` and `Option<i32>` are not the same type, and cannot be treated equivalently. We want to use option in the circumstances that the value it refers to could potentially not exist, thus forcing us to handle that scenario.

7.2 Match Control Flow

Match example

```
enum Coin {
        Penny,
        Nickel,
        Dime,
        Quarter,
}
fn value_in_cents(coin: Coin) -> u8 {
        match coin {
                Coin::Penny => 1,
                Coin::Nickel => 5,
                Coin::Dime => 10,
                Coin::Quarter => 25,
        }
}
```

Matches are useful for finding a matching component, binding a piece to something else, and executing based off of it. A further example of matching

```rust
fn value_in_cents(coin: Coin) -> u8 {
        match coin {
                Coin::Penny => 1,
                Coin::Nickel => 5,
                Coin::Dime => 10,
                Coin::Quarter(state) => {
                        println!("State quarter from {state:?}!");
                        25
                }
        }
}
```

If we passed `Coin::Quarter(UsState::Alaska)`, it would match as a quarter and state would take the value `UsState::Alaska`. This becomes incredibly useful with `Option<T>` as we can match whether the value is something or none. For example, the following checks if the input has a type of `Some(#)` or if it is `None` and execute code and return a value based on this. One important thing to note on matches is that they must be exhaustive. If we reach the end of a match and nothing has matched, we have a problem. If we want to catch the value at the end and use it, we match a variable we define.

```rust
match dice_roll {
        3 => add_fancy_hat(),
        7 => remove_fancy_hat(),
        1 other => move_player(other),
}
```

If you don't want to use this value, we use `_`, an underscore. This pattern will match anything but does not bind to it.

7.3 If Let

In the case that we want to have a single branch match, we can use an `if let`.

```rust
        let config_max = Some(3u8);
        if let Some(max) = config_max {
                println!("The maximum is configured to be {max}");
        }
```

In the above code, we are matching `config_max` to `Some(max)`. As some value exists, the value will match the max pattern and the following block will execute. Additionally, we can add an `else` to match the scenarios we would use an underscore pattern for.

7.4 Let...else

Let...else allows us to take action only if the pattern does not match, leading to a simpler flow when the desired match occurs.

```
        fn describe_state_quarter(coin: Coin) -> Option<String> {
                let Coin::Quarter(state) = coin else {
                        return None;
                };
        }
        if state.existed_in(1900) {
                Some(format!("{state:?} is pretty old, for America!"))
        } else {
                Some(format!("{state:?} is relatively new."))
        }
```

Index